

Algorithms: *Linear Search*

Linear Search

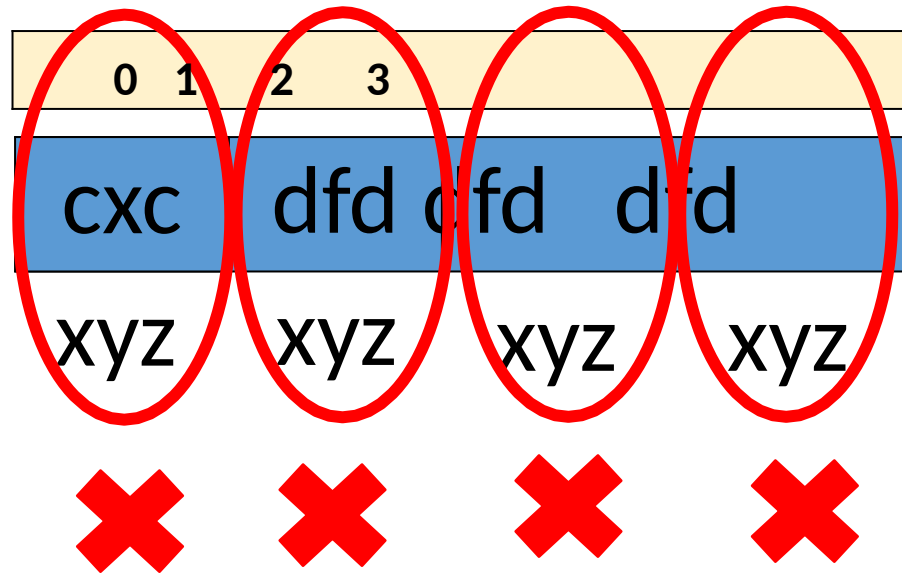
- The most basic searching algorithm
- Given an item to search from a bucket of items, we search ONE BY ONE.
- If we find then bingo! Else we look till the end of bucket.
- Example: Given an array of items. We will try to find “xyz” from it using linear search.

0	1	2	3
cxc	dfd	dfd	dfd

Linear Search

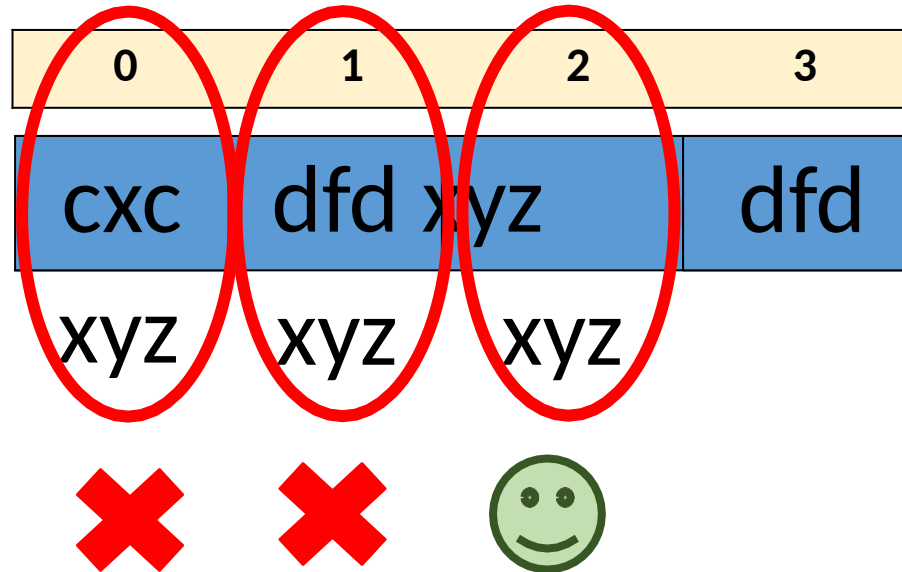
Simulation

As mentioned in the previous slide linear search is done one by one hence we start searching for “xyz” from index 0. If the items match we stop else we look for the next item in the array.



Linear Search

Simulation



Linear Search

Pseudo code (Iterative)

```
Boolean linearSearch(A[],item)
```

```
{
```

```
    int i = 0;
```

```
    while(i < A.length){
```

```
        if (A[i] matches item){
```

```
            return true;
```

```
        }
```

```
        i++;
```

```
    }
```

```
    return false;
```

```
}
```

Starting from index 0 of the array A

Loop to traverse till the end of the array

Checking if the *ith* element of the array matches with *item*

If matches, then return true, not searching further.

If mismatch in the if condition, this line will run moving to the next index of the array.

The program will reach this line after while loop is complete. This means all indices are traversed and item could not be found. False is returned.

Linear Search

Time Complexity (Iterative)

```
boolean linearSearch(A[], item){  
  
    int i = 0;  
    while(i < A.length){  
        if (A[i] matches item){  
            return true;  
        } i++;  
    }  
    return false;  
}
```



0	1	2	3
cxc	dfd	dfd	dfd
×	×	×	×

The worst case scenario occurs when the loop runs from index 0 to the end of the array. That is the item is found in the last index or not found at all. Recall the above scenario.

If there are n indices in an array, the worst case would be $O(n)$

Now for the best case to occur the item we are searching for appears at the beginning as a result the number of search is just 1. This is an once in a blue moon situation must be ignored.

Algorithms: *Binary Search*

Binary Search

- A divide and conquer algorithm
- Does not search the entire array like linear search
- Given an array of **sorted elements** and the item, k , to search for, we first check if the middle element matches k .
- If we find then bingo! Else we check $k > \text{middle element}$ or $k < \text{middle element}$? If the first condition is true we look into the right half of the array. If the second condition is true we look into the left half of the array.
- Note that the red marked conditions are true if and only if the array is sorted.

Binary Search

Pseudo code

(Iterative)

```
boolean binarySearch(A[], l, r, item){  
    while (l <= r){  
        mid = (l + r) / 2;  
        if (A[mid] == item){  
            return true;  
        }else{  
            if (item < A[mid]){  
                r = mid - 1;  
            }else{  
                l = mid + 1;  
            }  
        }  
    }  
    return false;  
}
```

l must always be less or equal to r for the loop to run

Find the mid index

Check if item matches with the mid element, return true if does else next step.

Check if the item is smaller, if yes shift the search range to left by moving r

If the above condition is dissatisfied, shift the search range to right by moving l

The program will reach this line after while loop is complete. False is returned.

Binary Search

Simulation

l

r

0	1	2	3	4
7	12	34	58	87

Search=87

```
boolean binarySearch(A[], l, r, item){
    while (l <= r){
        mid = (l + r) / 2;
        if (A[mid] == item){
            return true;
        } else {
            if (item < A[mid]){
                r = mid - 1;
            } else {
                l = mid + 1;
            }
        }
    }
    return false;
}
```

Binary Search

Time Complexity (Iterative)

In order to compute the time complexity of divide and conquer problems and focus on the problem size and work done at every step. Work done refers to finding the mid, matching, changing left or right. These are constant operations. Refer to the previous slide. You will see that at each the step problem size was reduced by factor of 2.

Initially the domain of search was the entire array of length n . In the second step we only considered the right half. The domain got reduced to $n/2$. In the following steps this problem was getting smaller by factor of 2; $n/4$, $n/8$, ... until 1 or 0. We need to count how many steps it took for the problem of size n to become 0 or 1 then multiply it by work done at each step.

Problem Size	Step No.
n	0
$n/2$	1
$n/4$	2
$n/8$	3
....	
1	k

Binary Search

Time Complexity (Iterative) Contd.

Problem Size	Step No.	Work done at each step
n	0	1
n/2	1	1
n/4	2	1
n/8	3	1
....		1
1	k	1

It took k steps for the searching to end and work done at each step was constant. We need to find k in terms of n . If you notice the divisors of the problem size (marked red) are all powers of 2 and we can use the step numbers as exponents. Therefore each problem size can be written as,

$n/2^{(\text{step no.})}$

The last line can be written as, **$1 = n/2^k$** .

If we solve it, we will find that **$k = \log_2 n$**

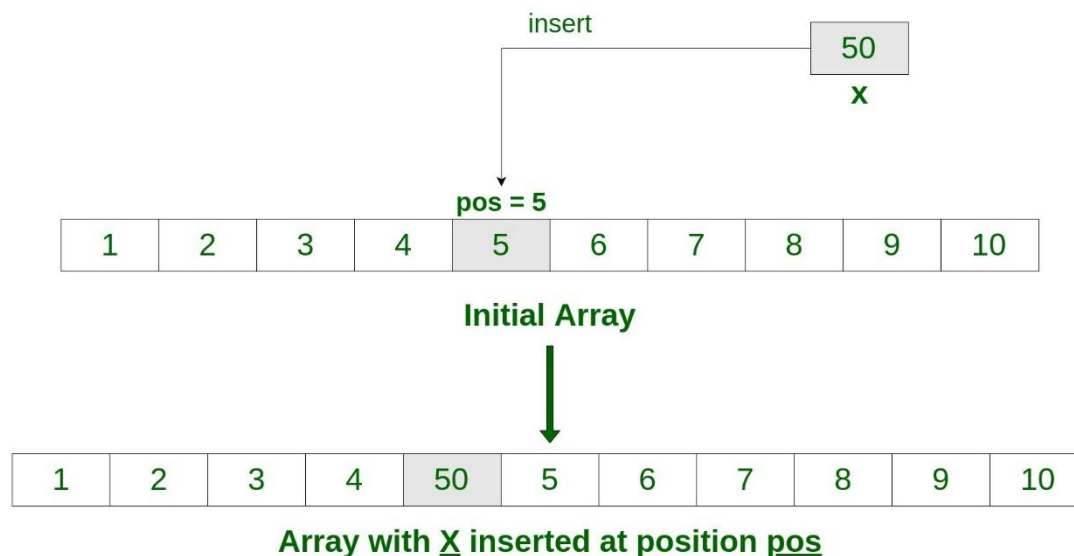
Therefore the time complexity is **$\log_2 n \times 1$** , which eventually is **$O(\log_2 n)$**

Problem

C program to Insert an element in an Array

Given an array `arr` of size `n`, this article tells how to insert an element `x` in this array `arr` at a specific position `pos`.

Insert an element at a specific position in an Array



Follow the below steps to solve the problem:

- First get the element to be inserted, say `x`
- Then get the position at which this element is to be inserted, say `pos`
- Then shift the array elements from this position to one position forward(towards right), and do this for all the other elements next to `pos`.
- Insert the element `x` now at the position `pos`, as this is now empty.

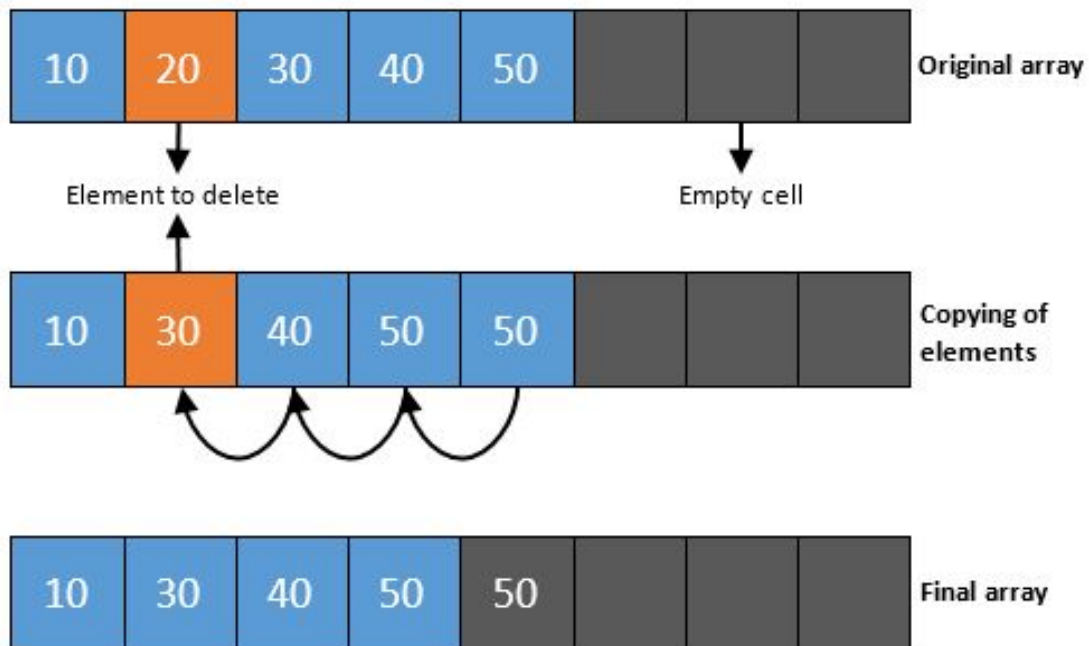
CODE

```
#include<stdio.h>
int main()
{
    int i,size,a[50],index,value;
    scanf("%d",&size);
    for(i=0;i<size;i++)
    {
        scanf("%d\n",&a[i]);
    }
    scanf("%d\n",&index);
    scanf("%d",&value);
```

```
    for(i=size-1;i>=index;i--)
    {
        a[i+1]=a[i];
    }
    a[index]=value;
    size+=1;
    for(i=0;i<size;i++)
    {
        printf("%d ",a[i]);
    }
    return 0;
}
```

Problem

Remove an element from an array in C



Step by step descriptive logic to remove element from array.

1. Move to the specified location which you want to remove in given array.
2. Copy the next element to the current element of array. Which is you need to perform $\text{array}[i] = \text{array}[i + 1]$.
3. Repeat above steps till last element of array.
4. Finally decrement the size of array by one.

CODE

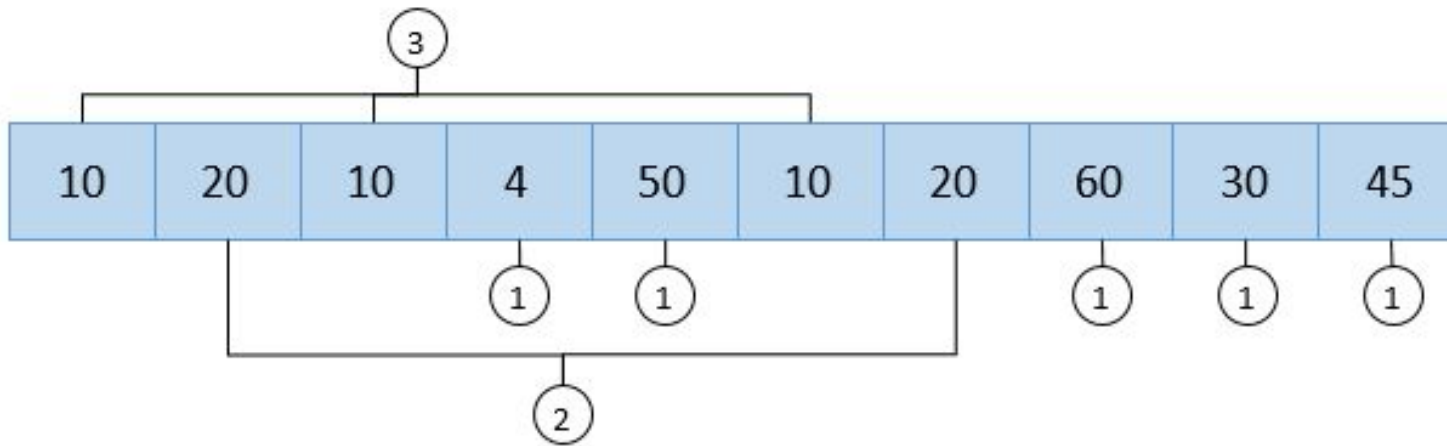
```
#include<stdio.h>
int main()
{
    int i,size,a[50],index;
    scanf("%d",&size);
    for(i=0;i<size;i++)
    {
        scanf("%d\n",&a[i]);
    }
    scanf("%d",&index);
```

```
    for(i=index;i<(size-1);i++)
    {
        a[i]=a[i+1];
    }
    size--;
    for(i=0;i<size;i++)
    {
        printf("%d ",a[i]);
    }
    return 0;
}
```


Practice!!

C program to count frequency of each element in an array

Write a C program to input elements in array and find frequency of each element in array. How to count occurrence of each element in array in C programming using loop. Logic to count frequency of each element in array in C program.



Example

Input

Input array elements: 5, 10, 2, 5, 50, 5, 10, 1, 2, 2

Output

Frequency of 5 = 3

Frequency of 10 = 2

Frequency of 2 = 3

Frequency of 50 = 1

Frequency of 1 = 1